

In a native compiler for C to machine language, the lowering from the high level language to machine language typically happens in two or more steps. Broadly, the program in high level language is lowered to an intermediate code first. This intermediate code is typically independent of the target machine, and various machine independent optimisations are performed on it. Then the intermediate language is lowered to a form more close to the target machine code. However, the intermediate code is usually internal to the specific compiler implementation and is not exposed outside. Also, the application binary interface specification, which deals with the application code making calls to the underlying operating system, is platform specific. Therefore, it is not possible to convert C to intermediate code and then recompile the intermediate code back to the target machine code on whichever platform we want to run. On the other hand, that's what gets facilitated by Java. The high level Java is translated to intermediate representation of byte codes. The byte code representation is well defined and exposed internally. The target machine on which the byte codes are expected to be executed is the Java Virtual Machine whose interface definitions and behaviour are well defined by JVM specifications.

Given what we now know, we can have abstract virtual machines built for different programming languages. We can classify the languages into static or dynamic, depending on whether they are converted to target machine code statically or they are translated to target machine code dynamically at runtime when the application is actually in the process of being executed. Languages like C/C++/Fortran are statically compiled languages, whereas Java, Haskell, Python or Lisp are dynamically translated languages. Note that I used the term 'dynamically translated' instead of dynamically compiled in the previous sentence. It is possible to use either an interpreter, a compiler or a combination of the two at runtime to translate a dynamic language to target machine code. For instance, in a Java Virtual Machine, it is possible to start off by interpreting the byte codes of methods and compile only those methods which are frequently called in the application. This is the technique adopted by Oracle's Hotspot JVM, which uses a combination of interpreter and compiler. The term 'Hotspot' stands for compiling only hot methods at runtime. Recall that irrespective of whether we do interpretation or dynamic compilation, the cost of translating to native machine code is incurred at runtime and adds to the application execution time. This is where traditional statically compiled binaries score over dynamically translated applications.

Statically compiled languages do not incur

any runtime cost for compilation to machine code. However, there are also various benefits to dynamically translated languages. For instance, it is possible to have greater flexibility in a dynamically translated language by being able to determine properties or generate new functionality dynamically, based on runtime data. As an example, it is possible to delay type checking to runtime instead of having to do type checking statically. On the other hand, it is also possible to have static type checking with a dynamically translated language. For instance, Java is a statically type checked language just as C is a statically type checked language. On the other hand, languages like Javascript, Python, Ruby, Lisp, etc, are dynamically typed in the sense that the majority of their type checking is performed at runtime. However, note that there have been recent proposals to support dynamic typing on Java Virtual Machine. The reason is that many of the popular Web languages like Python, Ruby, Javascript, etc can emit byte codes, and allowing them to run on JVM facilitates easier interaction. More details on dynamic type support in JVM can be found from JSR-292 available at <http://jcp.org/en/jsr/detail?id=292>. Also, note that many of the dynamically typed languages provide static type checking optionally, where it is possible.

Another dimension to consider is whether a language is strongly typed or whether it is weakly typed. Strongly typed languages dictate where the type associated with a block of memory is fixed; for inter-mixing operations with different types of operands, explicit type conversions are not needed by the programmer by means of casting, and no type conversion occurs implicitly. For instance, Java is a strongly typed language. It avoids errors due to incorrect implicit type conversions supported by the language. On the other hand, a weakly typed language is one that allows implicit conversions between types. C is a weakly typed language. One could have dynamically typed languages that are strongly typed, such as C# and Python. On the other hand, we could also have dynamically typed languages that are weakly typed, such as Perl or PHP. So strong/weak typing is a property orthogonal to static or dynamic typing.

Another property associated with languages supported over virtual machines is that many of them have support for automatic memory management, a.k.a. garbage collection. In a language like C, the programmer manually needs to free the memory dynamically allocated. On the other hand, in a dynamic language like Java, the VM provides automatic memory management facilities. A garbage collector is an important part of a virtual machine as it supports automatic memory management and the performance